



UNIVERSIDADE
LUSÓFONA

Mini-Projeto

Lista de Compras com Consulta Online

Gonçalo Colaço

José Pedro Moreira

Miguel Fernandes

Sandro Ribeiro

Linguagens de Programação | Engenharia Informática

www.ulusofona.pt

Índice

Índice	II
Índice de figuras	II
1. Introdução	1
1.1. Contexto	1
1.2. Enquadramento do Projeto	1
2. Descrição Geral da Aplicação	1
2.1. Visão Geral da Aplicação	1
2.2. Principais Funcionalidades	1
3. Tecnologias Utilizadas	2
3.1. Linguagem de Programação	2
3.2. Interface Gráfica	2
3.3. Base de Dados	2
3.4. API REST para Consulta de Preços	2
4. Arquitetura e Organização do Código	3
4.1. Estrutura do Projeto	3
4.2. Descrição dos Componentes	4
5. Base de Dados	5
5.1. Estrutura das Tabelas	5
5.2. Persistência e Histórico	5
6. Consulta de Preço através da API	6
6.1. Descrição da API	6
6.2. Comunicação entre a Aplicação e a API	7
7. Interface Gráfica	8
7.1. Conceção da Interface	8
7.2. Descrição da Interface	8
8. Testes e Validação	9
9. Conclusão	9

Índice de figuras

Figura 1 – Estrutura do projeto	3
Figura 2 – Interface da aplicação de planeamento de listas de compras	8

Lista de Compras com Consulta Online

1. Introdução

1.1. Contexto

O presente relatório descreve o desenvolvimento de uma aplicação para planeamento de listas de compras, realizada no âmbito da unidade curricular de Linguagens de Programação. O projeto visa aplicar, de forma prática, os conhecimentos adquiridos ao longo do semestre, através da conceção e implementação de uma aplicação funcional e estruturada.

1.2. Enquadramento do Projeto

A organização de listas de compras e a estimativa do respetivo custo são tarefas comuns que podem ser facilitadas através de ferramentas digitais. Neste contexto, foi desenvolvida uma aplicação que permite ao utilizador criar listas de compras, gerir produtos e consultar preços de forma automática.

A aplicação foi implementada em *Python*, integrando uma interface gráfica *Tkinter*, uma base de dados *SQLite* para persistência da informação e uma *API REST* para consulta de preços. Esta arquitetura permite simular um cenário real de comunicação entre uma aplicação cliente e um serviço externo.

2. Descrição Geral da Aplicação

2.1. Visão Geral da Aplicação

A aplicação foi concebida para funcionar de forma intuitiva, exigindo pouca interação do utilizador para realizar as tarefas principais, e garantindo simultaneamente a persistência e consistência dos dados. A estrutura da aplicação permite ainda uma utilização autónoma, criar e gerir automaticamente os dados necessários ao seu funcionamento.

2.2. Principais Funcionalidades

As principais funcionalidades implementadas na aplicação, são as seguintes:

- Seleção de produtos a partir de uma lista disponível.
- Definição da quantidade de cada produto.
- Adição e remoção de itens da lista de compras.
- Cálculo automático do custo total da lista.
- Limpeza completa da lista de compras.

- Gravação de histórico numa base de dados.
- Consulta detalhada de listas anteriormente criadas.
- Consulta de preços através de uma API.

3. Tecnologias Utilizadas

3.1. Linguagem de Programação

A aplicação foi desenvolvida em *Python*, uma linguagem de programação amplamente utilizada no ensino e no desenvolvimento de aplicações, devido à sua simplicidade, legibilidade e vastas bibliotecas nos diversos domínios.

3.2. Interface Gráfica

A interface gráfica da aplicação foi implementada utilizando a biblioteca *Tkinter*.

O *Tkinter* permitiu criar uma interface simples e intuitiva, composta por componentes como botões, caixas de seleção, tabelas e listas, facilitando a interação do utilizador com a aplicação. Foi ainda adotado um tema escuro, inspirado em aplicações modernas, com o objetivo de melhorar o conforto visual e legibilidade.

3.3. Base de Dados

Para a persistência da informação foi utilizada uma base de dados *SQLite*. Esta tecnologia foi escolhida por ser leve, fácil de integrar e adequada a aplicações de pequena e média dimensão.

A base de dados é criada automaticamente na primeira execução da aplicação e é utilizada para armazenar:

- As listas de compras criadas.
- Os itens associados a cada lista.
- O valor total de cada lista.

3.4. API REST para Consulta de Preços

A consulta de preços dos produtos é realizada através de uma *API REST*, desenvolvida no âmbito do projeto.

A *API* disponibiliza *endpoints* que permite obter a lista de produtos disponíveis e consultar o preço individual de cada produto. A aplicação gráfica comunica com a *API* através de pedidos *HTTP*, utilizando a biblioteca *requests*.

4. Arquitetura e Organização do Código

A aplicação foi desenvolvida seguindo uma organização modular, com o objetivo de separar responsabilidades e facilitar a compreensão, manutenção e evolução do código.

Cada componente do sistema foi isolado em ficheiros distintos, de acordo com a sua função específica. Esta abordagem permite uma estrutura clara, evitando a mistura de lógica de interface, lógica de negócio, acesso e dados e comunicação com serviços externos.

4.1. Estrutura do Projeto

O projeto encontra-se organizado em vários ficheiros e pastas, de forma a separar as diferentes responsabilidades da aplicação e facilitar a sua compreensão e manutenção.

O ficheiro *main.py* corresponde à aplicação principal, sendo responsável pela interface gráfica e pela interação com o utilizador. É neste módulo que são tratados os eventos da interface e coordenada a comunicação com os restantes componentes do sistema.

O ficheiro *modelo.py* contém as classes que representam o domínio da aplicação, nomeadamente os produtos, os itens de uma lista de compras e a própria lista. Este módulo concentra a lógica associada aos dados e aos cálculos realizados pela aplicação.

A persistência da informação é tratada no ficheiro *db.py*, responsável pela criação e gestão da base de dados *SQLite*, incluindo o armazenamento das listas de compras e dos respetivos itens.

A *API REST* encontra-se organizada numa pasta própria do projeto, contendo a implementação do serviço de consulta de preços e o ficheiro de dados dos produtos. Esta separação permite isolar o serviço de fornecimento de dados da aplicação gráfica.

Esta estrutura modular contribui para um código organizado, claro e alinhado com as boas práticas de desenvolvimento.

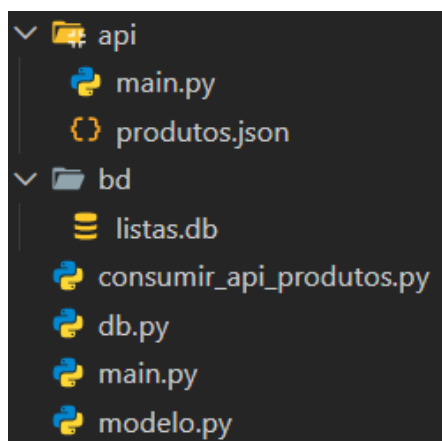


Figura 1 – Estrutura do projeto

Lista de Compras com Consulta Online

4.2. Descrição dos Componentes

Cada componente do sistema tem a sua respetiva responsabilidade. A interface gráfica comunica com os restantes módulos através de métodos específicos, sem aceder diretamente à base de dados ou à lógica interna da API.

A separação da API numa pasta própria permite isolar o serviço de fornecimento de dados da aplicação cliente, facilitando futuras alterações ou extensões do sistema sem impacto significativo na interface gráfica.

De modo geral, a arquitetura adotada contribui para um sistema coeso, legível e fácil de compreender e alterar.

Descrição

- **api/**: micro-API local em FastAPI

api/main.py: servidor FastAPI que serve produtos.json.

expõe as rotas `http://<IP>:8000/`, `http://<IP>:8000/produtos` e `http://<IP>:8000/produtos/{id}`.

api/produtos.json: ficheiro JSON com catálogo de produtos e preços.

produtos.json contém um catálogo de produtos (id, nome, preco, unidade).

- **bd/**: pasta onde é criado o ficheiro SQLite (via db.py).

- **modelo.py**: Classes de domínio (Produto, ItemLista, ListaCompras).

Produto: nome e preço.

ItemLista: associação produto + quantidade e cálculo do subtotal.

ListaCompras: coleção de itens e soma do total.

- **db.py**: Camada de persistência com SQLite (criação de tabelas, guardar/ler listas).

Usa `sqlite3` e guarda o ficheiro em `bd/listas.db` (caminho relativo a `db.py`).

Tabelas: `listas(id, data, total)` e `itens(id, lista_id, produto, quantidade, preco_unit)`.

Funções principais: `guardar_lista`, `obter_ultimas_listas`, `obter_itens_de_lista`.

- **consumir_api_produtos.py**: Cliente que consome a API de preços.

`GestorPrecos` obtém a lista de produtos ao consultar `http://<IP>:8000/produtos`.

Converte a resposta JSON num dicionário `{"nome (unidade)": preco}`.

Métodos: `listar_produtos()` e `obter_preco(nome)`.

Lista de Compras com Consulta Online

- **main.py:** Aplicação Tkinter (UI + lógica de interação).

UI construída com tkinter e ttk.

Interação com GestorPrecos para listar produtos e obter preços.

Mantém ListaCompras em memória e atualiza Treeview e total.

Operações: adicionar, remover, limpar, guardar (persistir no BD), ver histórico.

5. Base de Dados

Para possibilitar o registo de listas de compras bem como os seus respetivos itens e valores totais, a aplicação utiliza uma base de dados SQLite, adequada a aplicações de pequena e média dimensão e de simples integração com Python.

5.1. Estrutura das Tabelas

A base de dados é composta por duas tabelas principais:

A tabela *listas* é utilizada para armazenar a informação geral de cada lista de compras, incluindo um identificador único, a data de criação e o valor total associado à lista.

```
listas(id, data, total)
```

A tabela *itens* armazena os itens pertencentes a cada lista de compras, incluindo a designação do produto, a quantidade e o preço unitário registado no momento da gravação. Cada item encontra-se associado a uma lista específica através de uma relação entre as tabelas.

```
itens(id, lista_id, produto, quantidade, preco_unit).
```

5.2. Persistência e Histórico

Sempre que o utilizador guarda uma lista de compras, os dados são registados na base de dados, permitindo a sua consulta posterior. A aplicação disponibiliza um histórico de listas guardadas, possibilitando ao utilizador visualizar listas anteriormente criadas e os respetivos detalhes.

Esta abordagem assegura a consistência dos dados e permite manter um registo organizado das listas de compras ao longo do tempo.

Lista de Compras com Consulta Online

6. Consulta de Preço através da API

6.1. Descrição da API

A API REST disponibiliza endpoints que permitem obter a lista de produtos disponíveis e consultar o preço individual de cada produto. A informação é fornecida em formato JSON, facilitando a sua interpretação e integração na aplicação cliente

A implementação da API encontra-se organizada numa pasta própria do projeto, garantindo uma separação clara entre o serviço de fornecimento de dados e a aplicação principal.

```
from fastapi import FastAPI
import json
import uvicorn ##a
import socket
import os

app = FastAPI(
    title="API de Preços de Produtos",
    description="API simples para disponibilizar preços de produtos em JSON",
    version="1.0.0"
)

path_file = os.path.dirname(__file__) #pasta do ficheiro python
path_file_json = os.path.join(path_file, "produtos.json")

try:
    with open(path_file_json, 'r', encoding='utf-8') as file:
        produtos = json.load(file)
except:
    print("Erro ao abrir o ficheiro.")
else:

    hostname = socket.gethostname()
    IPAddr = socket.gethostbyname(hostname)

    @app.get("/", tags=["Geral"])
    async def root():
        """Rota inicial de boas-vindas."""
        return [{"mensagem": "Bem-vindo à API de Produtos!"},
                {"rotas": "http://" + IPAddr + ":8000/produtos | http://" + IPAddr + ":8000/produtos/id"}]

    @app.get("/produtos")
    def listar_produtos():
        return produtos

    @app.get("/produtos/{produto_id}")
    def obter_produto(produto_id: int):
        for produto in produtos:
            if produto["id"] == produto_id:
                return produto
        return {"erro": "Produto não encontrado"}

# Bloco para execução
if __name__ == "__main__":
    # O servidor ficará disponível em http://127.0.0.1:8000
    # uvicorn.run(app, host="127.0.0.1", port=8000)
    uvicorn.run(app, host=IPAddr, port=8000)
```

Lista de Compras com Consulta Online

6.2. Comunicação entre a Aplicação e a API

Durante a utilização da aplicação, sempre que o utilizador seleciona um produto, é efetuado um pedido à API REST para obtenção do respetivo preço.

A resposta recebida é processada pela aplicação gráfica e utilizada no cálculo dos subtotais e do custo total da lista de compras.

A comunicação é realizada através de pedidos HTTP, recorrendo à biblioteca requests.

```
import json
from pathlib import Path
import socket
import requests

# Consumir a API de produtos
class GestorPrecos:

    def __init__(self):
        hostname = socket.gethostname()
        IPAddr = socket.gethostbyname(hostname)

        url = f"http://{IPAddr}:8000/produtos"
        response = requests.get(url)

        if response.status_code == 200:
            produtos = response.json()
            #print(produtos)

            # converter lista de dicionários para dicionário de preços
            produtos = {p["nome"] + " (" + p["unidade"] + ")": p["preco"] for p in produtos}

            self.dados = produtos

        else:
            print("Erro ao aceder à API")

    def obter_preco(self, nome_produto: str) -> float | None:
        valor = self.dados.get(nome_produto)
        return float(valor) if valor is not None else None

    def listar_produtos(self) -> list[str]:
        return sorted(self.dados.keys())
```

Lista de Compras com Consulta Online

7. Interface Gráfica

7.1. Conceção da Interface

A interface gráfica da aplicação foi desenvolvida com o objetivo de proporcionar uma utilização simples e intuitiva. A disposição dos elementos permite ao utilizador realizar as principais operações de forma direta.

A aplicação apresenta um tema escuro, inspirado em aplicações modernas, com o objetivo de melhorar o conforto visual e a legibilidade a utilização prolongada.

7.2. Descrição da Interface

A figura 2 apresenta a interface da aplicação. Nesta interface encontram-se os principais componentes de interação, nomeadamente a área de seleção de produtos e quantidades, a lista de compras em construção, o valor total calculado automaticamente e a secção de histórico de listas guardadas.

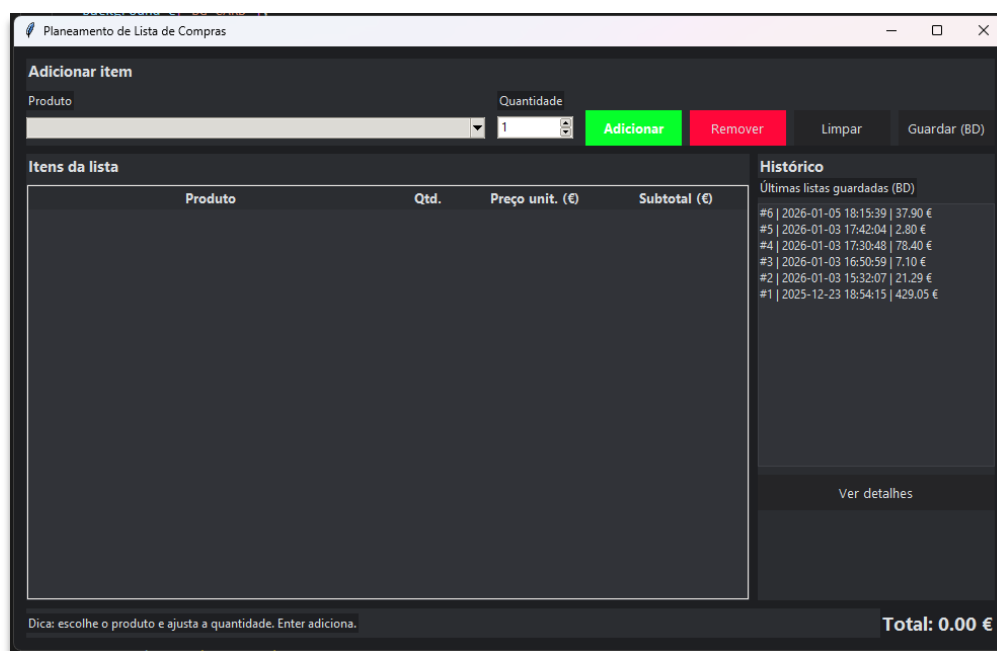


Figura 2 – Interface da aplicação de planeamento de listas de compras

Lista de Compras com Consulta Online

8. Testes e Validação

De forma a garantir o correto funcionamento da aplicação, foram realizados testes funcionais manuais ao longo do desenvolvimento do projeto. Estes testes incidiram sobre as principais funcionalidades disponibilizadas ao utilizador.

Foram testados, entre outros, os seguintes aspetos:

- Adição e remoção de produtos da lista de compras.
- Cálculo automático dos subtotais e do valor da lista.
- Gravação de listas de compras na base de dados.
- Consulta do histórico de lista guardadas.
- Comunicação com a *API REST* para obtenção de preços.

Os testes realizados permitiram verificar que a aplicação responde corretamente às opções do utilizador e que os dados são tratados de forma consistente, garantindo um funcionamento estável do sistema.

9. Conclusão

O projeto desenvolvido permitiu a implementação de uma aplicação funcional para planeamento de listas de compras, integrando uma interface gráfica, uma base de dados e uma *API REST* para consulta de preços.

Ao longo do desenvolvimento foi possível aplicar, de forma prática, os conhecimentos adquiridos na unidade curricular de Linguagens de Programação, resultando numa solução organizada, coerente e alinhada com os objetivos do trabalho proposto.